Augmenting Large Language Models with Retrieval-Augmented Generation for Contextualized Academic Assistance

Evan Loo
University of Southern California
evanhsie@usc.edu

Matthew Jiang
University of Southern California
jiangmy@usc.edu

1 Introduction

This project is focused on creating a contextualized Question and Answering (Q&A) chatbot that can help users navigate a database of texts. Specifically, this chatbot is designed for general students' use – studying for an academic course with a lengthy textbook to read, or numerous documents of class notes to sift through. Here, our project is intended as a proof of concept, hence we trained the chatbot on actual texts taken from our CSCI467 class on Introduction to Machine Learning and CSCI444 class on Natural Language Processing. We chose these texts as they were readily available from when we previously took the classes, and we were confident in our relative expertise in this computer science subfield to accurately evaluate the model's responses.

Delving deeper into the chatbot, it uses Retrieval Augmented Generation (RAG) to contextualize its responses to students' database of texts, and it is further coupled with a Large Language Model (LLM) to provide a fully natural language chatting experience. Leveraging the RAG capabilities, the chatbot response will also include page-level links so users can easily navigate the database, locate the section of text referenced by the chatbot, and validate its responses. Furthermore, the natural language aspect of the RAG+LLM model means that it is designed for use by everyday users with no technical knowledge. Instead of highly syntactical SQL queries to navigate a database of texts, users simply type in their questions about a certain piece of knowledge stored in the texts, and the chatbot does all the search legwork, then summarizes the data retrieved back into natural language in its response to the user.

But zooming out from our RAG+LLM project, one might wonder how this could be better than the current state-of-the-art LLMs like ChatGPT that have amassed hundreds of millions of users to date. Unfortunately, this highly generalized nature of LLMs becomes a disadvantage when answering questions on a single topic as a dedicated study chatbot. Here, the LLMs are prone to more general and less in-depth answers, and also lack citations for their answer sources. These limitations are a crucial aspect for when studying and trying to locate excerpts directly from a textbook, hence why the addition of RAG will greatly enhance the LLM's contextual capabilities and overall usability.

On that note of making LLMs more usable, a secondary goal of our study chatbot is intended to promote the democratization of LLMs to the general public. With the minor customization of an RAG, we aim to highlight the extensive range of the applications that an LLM can serve in everyday tasks. However, this democratization does require significant constraints as everyday people will not have access to the high performance computing hardware to locally run cutting edge LLMs. Instead, our project should be runnable on a standard Personal Computer and still compile within a reasonable amount of time (1-2 hours). Further, our project is designed to be used at no monetary cost, meaning we will only use open source LLMs (we chose Llama 3.2 with 8b Parameters), coupled with our self-designed RAG. Yet, in the face of these constraints, successfully completing our project would be a significant milestone for the democratization of LLMs that they can be customized to augment our everyday life. But of course, our project impact is most relevant to students who will gain a chatbot they can use 24/7 and free of charge for help with their studies.

2 Related Work

For our project goal of creating a chatbot to help students study, one alternative is standalone LLMs models. LLMs were the hallmark advancement in the field of natural language processing (NLP) that realized the idea of fully natural language

chatbots. Amongst the early models that gained mainstream popularity were GPT-3 (Brown et al., 2020) and BERT (Devlin et al., 2019). The primary breakthrough came from the introduction of bi-directional transformers that are trained on massive linguistic data sets to learn which words in the sentence to pay attention to. And with this architecture, LLMs are highly capable at maintaining a coherent and relevant conversation – a key requirement when interacting with a study chatbot. However, while the significant training data set enables the LLM to answer a diverse range of queries, it can also muddy the LLM's response from focusing on the single subject that students are studying for. Further, the cost of re-training the LLM to emphasize a specific field of knowledge would be exorbitant and impossible for our project constraints. Hence, our supplementary RAG component is necessary to build on the extensive Q&A capabilities of LLMs and restrict the overall model to only respond with respect to the database of texts the RAG is pulling from. Another common pitfall of LLMs is their tendency to hallucinate as mentioned in the Jurafsky textbook (Jurafsky and Martin, 2000) and research papers like *On Faithfulness and Factuality in Abstractive Summarization* by Maynez et al. (2020), LLMs generate hallucinated content – plausible-sounding but factually inaccurate information due to the model's reliance on probabilistic predictions to generate the next word, so it doesn't always align with factual accuracy. This is exacerbated when LLMs are used as a study chatbot as students are in the process of learning new knowledge, and will struggle to detect hallucinations in the LLM's responses. Instead, using a RAG+LLM model will constrain the model's responses to the contexts chosen by the RAG and reduce the likelihood of hallucinations. And even if hallucinations occur, the model's responses will include direct links to the page and document where the model retrieved its contexts. So validating the output is quick and efficient even for learning students, and improves the overall trustworthiness of the RAG+LLM.

Overall, the comparison between RAG+LLM and LLM-only models is a highly relevant topic in today's society where LLM-only models like GPT-4o and Copilot are widely used. And having posited numerous advantages from complementing LLMs with a RAG, we plan to investigate the difference between these 2 models in our testing campaign, as detailed in the Hypotheses section.

Moving on, we should also evaluate similar research done in the RAG+LLM realm. An example is BioRAG (Wang et al., 2024, BioRAG). This research trains its RAG embedding model on 22 million biological and medical scientific papers. Then during the testing phase, the model receives user queries and pre-processes it. Here, the query is deconstructed to find a closed topic tag using the Medical Subject Headings (MeSH) mapping. Additionally, the RAG is also run in a loop where the information retrieved is self-evaluated at the end. If the information is determined to be inadequate, the RAG re-runs to try and retrieve more adequate and relevant information. Using these techniques, the paper was successful at improving the performance over baseline LLM-only models like GPT3.5, as well as other scientific RAG frameworks like SciRAG that were significantly larger than BioRAG. However, the extensive finetuning of the model to its specific biomedical field also presents a downside as it will require significant re-training to contextualize it to a different field of knowledge. Not to mention, the re-training process itself requires extensive knowledge hierarchies like the Medical Subject Headings (MeSH) mapping that might not be readily available in other fields. From the perspective of our project, flexibility of the RAG component is a key part of our design, given our goal of using it to augment students' learning process, for any academic courses they might be studying. Further, the standard sources of information available to students are their course textbook, and possibly some supplementary notes or course slides.

Indeed, our RAG+LLM model is somewhat of a middle ground between the highly generalized LLMs like ChatGPT, and finely tuned models like BioRAG – these are requirements that are similar yet different enough to require a unique new approach.

3 Methods and Experiments

3.1 Hypotheses

3.1.1 Hypothesis 1: RAG+LLM model will perform better than the LLM-only model on various query formats such as true/false questions and open-ended questions

- **Independent Variable**: The type of model used to parse the test prompts (RAG+LLM vs.

LLM-only)

- **Dependent Variable**: The accuracy of the model's responses

This hypothesis is the premise underpinning our project to promote the democratization of LLMs. By complementing LLMs with RAG, we believe that the LLM hallucinations can be mitigated and it more accurately identifies the prompt context. Consequently, this should result in a significant improvement in the model performance that hopefully persuades students and the general public to integrate it into their everyday lives.

Note that "accuracy" is selectively measured to exclude the instances where the RAG fails to return the contexts relevant to the initial prompt. This isolates the accuracy metric to only compare the performance of the LLM – when it is constrained to the RAG's chosen contexts, and when it is not.

3.1.2 Hypothesis 2: For the 2 RAG+LLM models, Sentence Bidirectional Encoder Representations from Transformers (SBERT) vectorizer will perform better than Term Frequency Inverse Document Frequency (TF-IDF) vectorizer

- **Independent Variable**: The type of vectorizer used for the RAG+LLM model

- **Dependent Variable**: Accuracy and Relevance of model's responses and sources

TF-IDF is a long-standing metric used in the Information Retrieval space. It aims to identify "higher-value" words that are uncommon in the overall collection of text documents, but frequently-occurring in the current text document. It was first conceived by Karen Spärck Jones in her 1972 paper (Spärck, 1972), and its most significant application nowadays is in internet search engines. Overall, we chose this metric as we needed to distill contexts into 128 dimensions for quick RAG processing, and TF-IDF would capably identify the most representative and meaningful words to be used. Further, TF-IDF does not require additional storage space for word embeddings unlike the SBERT vectorizer, and it is a simple formula that requires minimal computational power. (Both these elements are a significant concern given the project's hardware constraints).

On the other hand, SBERT is a relatively newer approach that vectorizes sentences by leveraging the well-known transformer architecture to embed natural language sentences as vectors. For our implementation, we used pre-trained Global Vectors for Word Representation (GloVe) word embeddings (Pennington et al., 2014) that had a vocabulary size of 400000. (The embedding text file is 6GB in size, and we found it was the maximum size possible to still compile the program within the reasonable 1-2 hours).

The absence/presence of these pre-trained word embeddings were the primary difference between the TF-IDF and SBERT vectorizer approaches. We argue that this would enable the SBERT vectorizer to integrate a semantic interpretation of the contexts that was lacking when only considering word frequencies with TF-IDF. Accordingly, we also chose GloVe over other popular word embedding such as word2vec because of the GloVe training approach – "to learn word vectors such that their dot product equals the logarithm of the words' probability of co-occurrence". We believed that the emphasis on co-occurrence could lead to greater nuance when embedding the semantic meanings of contexts, further emphasizing the difference between SBERT and TF-IDF.

Ultimately, this was the basis for our hypotheses that SBERT would outperform TF-IDF in accuracy and relevance. Note that accuracy is also selectively measured as aforementioned, while relevance is the ratio of test cases where the RAG returns relevant contexts, as determined by the LLM when choosing to abstain.

3.2 Architecture

Here is a brief overview specifications on the base RAG+LLM model setup for future reference to replicate our study chatbot.

3.2.1 RAG

The RAG parses text documents (PDFs) into text strings, which are further chunked down into a more manageable size (800 characters – small chunks, but still large enough to preserve the meaning in the overall chunk). These chunks are vectorized (using TF-IDF or SBERT) into embeddings. These embeddings and will later be pulled as the contexts the LLM will use to better contextualize its response.

3.2.2 LLM

Based on the human query, the top (K=5) most relevant vector embeddings will be identified. These

will be used to format the LLM prompt, instructing the LLM to only answer the human query with respect to the context of the given K vector embeddings. Finally, this prompt is given to the LLM model (Llama 3.2, 8b Parameters), and the LLM response is the output returned to the user.

3.3 Experimental Protocol

Datasets and questions are split into groups with IDs '444' and '467'. We have folders named <ID>_data and <ID>_questions, for the PDF sets and question sets, respectively.

Embedding functions get ID's 'nomic', 'sbert', and 'tfidf'

To measure the performance of the RAG+LLM pipeline, we use the following methods for each question type:

1. True/False Questions: instruct the answering LLM to output either the word "TRUE" or "FALSE" and parse the output to determine its deviation from the expected

2. Short Answer Questions: utilize a second validation LLM to subjectively judge whether or not the output is accurate as similar to human judgment as possible.

We utilize a Bash script that automates the running of our pipeline. It first runs `populate_database.py` to populate the chroma database files with the pdfs from each test set. It then runs `validate_responses.py` with the embedding function ID as an argument.

This script automates the testing process, only requiring the user to enter a dataset ID and embedding function ID as arguments. A simple testing example command is as follows:

```
bash run_script.bash 444 tfidf
```

This script automates the testing process, making it easy to run and record large volumes of test sets and test data.

3.3.1 Validation LLM Verification

Because we are familiar with content contained in the CSCI 467 and CSCI444 datasets, we are able to use our expertise to manually verify the accuracy of the Validation LLM. Our process in verifying the validity of our Validation LLM is further described in Section 4.1 and Section 4.2.

3.3.2 Logging

We utilize the Python `logging` module to record and save large numbers of detailed logs for later reference. Log files are saved to a folder `/logs` and contains the following test details:

- Question Index, `int`
- Question, `string`
- Expected Answer, `string`
- Response, `string`
- Matches, `boolean`
- Relevant Sources, `List`

3.4 Finetuning

Here, after compiling our initial test suites, we noticed several issues that we attempted to finetune:

3.4.1 Interactive Command Line Interface

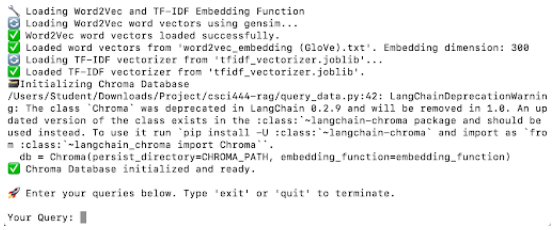

Figure 1: Interactive Command Line Interface

First, the RAG+LLM models were requiring a significant amount of time to compile, particularly when using the SBERT vectorizer which needed to load the 6GB word embeddings file. So we modified the program design to use an Interactive Command Line Interface 1 that would allow users to query the model multiple times in succession without having to re-run the program each time. This did lead to an increased compilation time when the program first initialized (still under 5 minutes), however the query times were shortened (10 seconds). And overall, this significantly accelerated our testing process as we tested 30 or more queries in each test run.

3.4.2 Prompt Engineering

Secondly, we observed instances where the LLM was mistakenly abstaining from answering the test prompts because it evaluated that the contexts chosen by the RAG were not relevant to the prompt. But upon closer inspection of the prompts and

RAG-selected contexts, we realized that the contexts were the contexts were relevant conceptually, but not directly applicable to the query. Figure 2 is one such test prompt that asks about 8-puzzles (Figure 4). However, the database of texts never mentioned 8-puzzles, only 15-puzzles which the RAG was able to correctly retrieve. But because our original LLM prompt was narrowly formatted to only use the text chunks as contexts, the LLM abstained – "there is no specific problem about the 8-puzzle given in the context. . . " (Figure 3).

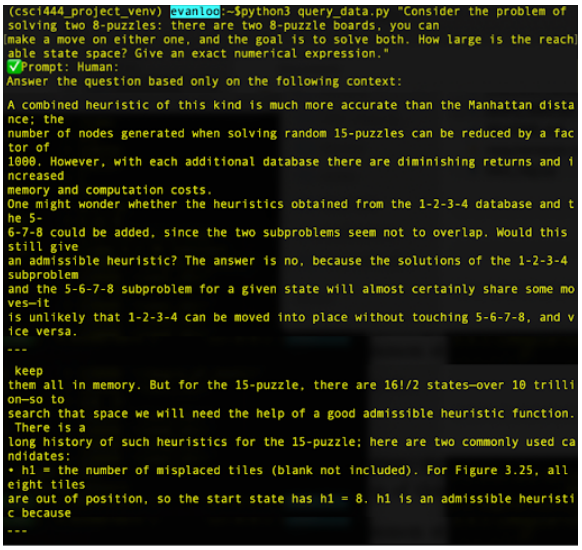

Figure 2: Test prompt about 8-puzzles, and a portion of the contexts retrieved by the RAG.

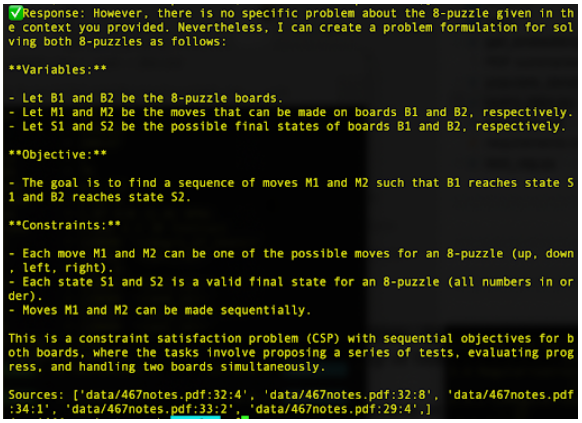

Figure 3: RAG+LLM model's response abstaining from answering the 8-puzzle prompt because it was not given relevant contexts.

Thus to resolve this issue, we finetuned multiple iterations of our LLM prompt and finally settled on modifying it to "Answer the question based only on the following context/conceptual example". And based on our final test suite, this prompt

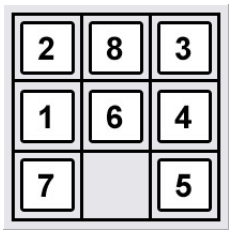

Figure 4: 8-puzzle – goal is to move the tiles to achieve a specific tile order.

engineering fix did work for prompts like the 8-puzzle one mentioned above. Notably, this solution is somewhat based on heuristics as we chose to train our models on the Computer Science subject. Consequently there were more application-focused, "novel" questions that cannot be retrieved directly from the textbooks content, so our study chatbot was adapted to handle these questions as well.

4 Results and Conclusions

4.1 Credibility of the Validation LLM

To determine the most accurate LLM for validating the RAG+LLM model, we tested several publicly available LLMs on a subset of 30 test cases. The results are summarized in the table below:

Model	P(Accurate)
llama3.2	0.71
hermes3	0.70
mistral	0.74
gpt4o-mini	0.95

We selected OpenAI's **gpt4o-mini** model to validate our pipeline output. The small cost associated with querying this model is negligible relative to the cost of the RAG+LLM pipeline, as it does not affect the user's operational cost.

4.2 Statistical Analysis of Validation Data

To quantify the potential error in our results, we compute the confidence interval (CI) at a 95% confidence level, assuming a validation accuracy $A_v = 0.95$ and $n = 30$ test cases. The confidence interval is calculated as follows:

$$\text{CI} = A_v \pm z(0.95) \times \sqrt{\frac{A_v \times (1 - A_v)}{n}}$$

Substituting the values:

$$\text{CI} = 0.95 \pm 1.96 \times \sqrt{\frac{0.95 \times (1 - 0.95)}{30}}$$

$$\text{CI} = 0.95 \pm 0.078$$

$$\text{CI} = [0.872, 1.028]$$

Using the lower bound of the confidence interval, we relate the true accuracy A_t to the measured accuracy A_m as follows:

$$A_m = 0.872 \times A_t + (1 - 0.872) \times (1 - A_t)$$

Rearranging for A_t, we obtain:

$$A_t = \frac{A_m - (1 - 0.872)}{0.872}$$

This result indicates that as long as the measured accuracy $A_m > 0.5$, the measured accuracy underestimates the true accuracy A_t. Therefore, our accuracy estimates are conservative.

4.3 Comparison of RAG plus LLM Model Performance on Different Datasets

We evaluated the RAG+LLM pipeline on two datasets with distinct content types:

- **CSCI 467:** A LaTeX-typeset PDF containing primarily text and neatly formatted equations.
- **CSCI 444:** A set of slide decks converted to PDF format, featuring a mix of plain text, equations, and images without captioned context.

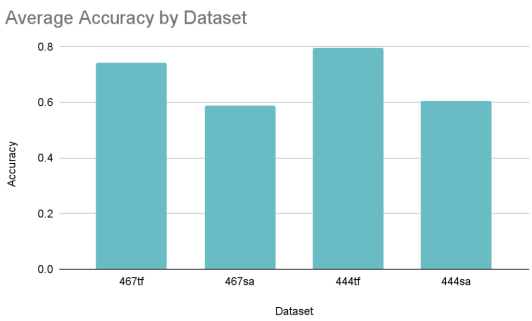

Figure 5: Performance comparison of the RAG+LLM model on CSCI 467 and CSCI 444 datasets, split into true/false and short answer questions.

As shown in Figure 5, the model performed better on true/false questions compared to short answer questions, and its accuracy was higher on the CSCI 467 dataset.

One possible explanation is that the **CSCI 467** dataset contains more text-based information, which makes it easier for the embedding functions to retrieve relevant contexts. In contrast, the **CSCI 444** dataset includes a larger proportion of images and equations with limited surrounding text, which hinders context retrieval.

4.4 Hypothesis 1: RAG+LLM model will perform better than the LLM-only model

First, the raw data we collected to investigate hypothesis 1 was the average accuracy of each embedding model, as shown in Figure 6:

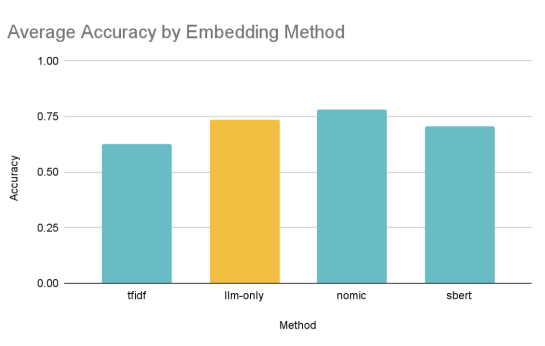

Figure 6: LLM-only performance compared to RAG+LLM performance. Questions where the LLM abstained are counted as incorrect.

Here, TF-IDF achieved an accuracy score of 0.62, and Sentence-BERT achieved the accuracy score of 0.70. Neither of these beat the LLM-only score of 0.73.

4.4.1 What About Nomic?

Nomic refers to the *nomic-embed-text* embedding method from Ollama, an open-source model repository. Nomic was used as a baseline to demonstrate the potential performance of a well-tuned open-source embedding model, as compared to our custom embedding functions implemented from scratch. However, Nomic was not included as part of any of our hypotheses.

4.4.2 Understanding The Data

Indeed, at first glance, the data appears to show that the addition of the RAG component harmed the model's performance. However, this raw accuracy metric is mis-representative of the models' true accuracy, as it categorizes test questions where the LLM abstained from answering as incorrect, thus masking the true benefit of the RAG+LLM pipeline.

However, the inability of the RAG to retrieve relevant contexts contributes to these abstentions. This is because the LLM prompt explicitly prevents

the model from answering questions when the necessary context is missing. And to fully understand the effect of context retrieval on performance, Figure 7 presents the relevance score for each RAG embedding method, defined as the proportion of test questions for which the LLM did not abstain due to missing context.

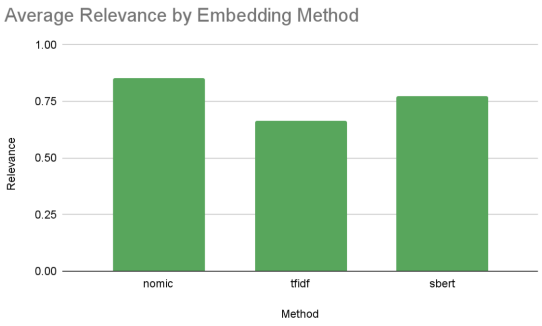

Figure 7: Relevance score: the proportion of test questions where the LLM did not abstain due to missing relevant context.

Now, to properly observe the performance difference between the RAG+LLM and LLM-only models, we use conditional accuracy to only compare the test cases where the RAG+LLM pipeline did not abstain from answering to the performance of the LLM-only model.

Figure 8 shows all metrics side-by-side. Two key metrics are:

- non-abstention accuracy (Average accuracy from Figure 6)
- Conditional accuracy, $P(\text{Accuracy} \mid \text{Relevance})$, which is accuracy given that the relevant context was successfully retrieved.

As expected, the conditional accuracy for all 3 RAG+LLM models significantly improved over their original non-abstention accuracy.

Also, Conditional relevance, ($P(\text{Relevance} \mid \text{Accuracy})$) shows the likelihood that the correct context was retrieved given that a test question was answered correctly. This value is expected to be equal to 1 with all methods, as the model is instructed to abstain from answering without relevant context. Given that our average conditional relevance for all embedding methods is 1, figure 8 shows that the answering LLM performs as instructed.

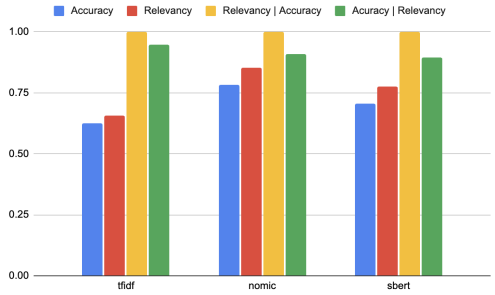

Figure 8: Metrics comparison: Non-abstention accuracy, conditional accuracy, conditional relevance.

4.4.3 Hypothesis 1 Conclusion

For clarity, Table 1 places the RAG+LLM's conditional accuracies alongside the LLM-only model accuracy to clearly compare their respective performances. This comparison reveals that the RAG+LLM model achieves higher accuracy when relevant contexts are retrieved, validating our hypothesis 1.

Model	$P(\text{Accurate})$
LLM Only	0.737
nomic	0.908
tf-idf	0.949
sbert	0.895

Table 1: Average accuracy for LLM-only model and Conditional accuracy for RAG+LLM models

4.5 Hypothesis 2: SBERT vectorizer will perform better than TF-IDF

The initial intuition behind this hypothesis is that the Sentence-BERT vectorizer incorporates contextual embeddings of words, unlike TF-IDF.

We measure the performance of the RAG embedding methods using two key metrics: (1) the Relevance score of each embedding method (i.e., the proportion of test cases where RAG successfully retrieved the correct context for the question), and (2) Conditional accuracy of the LLM when a relevant context was retrieved (i.e., the LLM did not abstain due to irrelevant context retrieval).

As shown in Figure 7, Sentence-BERT achieved the a higher relevance score of 0.71 compared to TF-IDF. Then, when examining the Conditional accuracy scores in Figure 8, we did not observe a statistically significant improvement in accuracy between the two embedding methods. This could be because when both embedding methods successfully retrieve relevant contexts, the contexts are equally relevant. So for conditional accuracy that

excludes instances of irrelevant context retrieval, the LLM responds equally accurately.

In conclusion, Hypothesis 2 is partially supported by our tests – SBERT did achieve a better relevance score than TF-IDF, but there was no significant variation in the overall models' conditional accuracy.

5 Future Work

Future improvements for our methodology may include:

- Testing and gathering data on more embedding techniques, such as fine-tuned transformer models or hybrid approaches
- Extending the RAG to also handle multimodal content, including images, charts, and equations, to improve performance on slide decks and visually dense materials
- Implementing dynamic chunking strategies to adaptively optimize context window size based on query complexity and document structure
- Enhancing the retrieval process with reranking algorithms to prioritize the most relevant contexts before passing them to the LLM

Additionally, We are considering incorporating an additional evaluation method to reduce the dependence on manual evaluations. This might involve a model similar to the Semantic Textual Similarity (STS) project developed by our classmates that evaluates the similarity between 2 sentences. For our purpose, these 2 sentences would be the expected and outputted answer by the RAG+LLM/LLM-only model. Then similar to our current validation LLM, we would expect a True/False response on whether the answers are sufficiently similar (Compared against a specific similarity threshold). Here, the benefit of using a tool specifically designed for measuring sentence similarities is an improved evaluation accuracy, not to mention the reduced costs over the current GPT-4o evaluation LLM. But ideally, these 2 (or more) evaluation tools could be used in tandem to the extent that it is statistically improbable that the evaluations are mistaken. Thus, manual evaluation is only required when there is an evaluation discrepancy. And overall, this automation will greatly improve the scalability of our project – we would have the capability to test larger test suites for statistically significant results.

Lastly, an improvement that could be built on this project is to implement of multi-modal processing, enabling the system to effectively handle not only text but also images, diagrams, and equations. This would allow the model to parse more diverse datasets, including math PDFs and slide decks. This enhancement would significantly broaden the applicability of the model, particularly in fields where visual data is integral to understanding, such as STEM disciplines and technical documentation.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding.

Daniel Jurafsky and James H. Martin. 2000. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, 1st edition. Prentice Hall PTR, USA.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, Online. Association for Computational Linguistics.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Spärck. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*.

Chengrui Wang, Qingqing Long, Meng Xiao, Xunxin Cai, Chengjun Wu, Zhen Meng, Xuezhi Wang, and Yuanchun Zhou. 2024. Biorag: A rag-llm framework for biological question reasoning.